

NIRIKSHAK

A self-learning, vendor-agnostic network security compliance auditor

Concept and Solution Report | Smart India Hackathon 2026 | Problem Statement 26155 | National Technical Research Organisation (NTRO) | Team Atlantis

1. The problem statement

PS 26155 asks for an AI-augmented, vendor-agnostic compliance engine for network device configurations. The background it sets out is specific, and the difficulty sits in two places that are easy to underestimate.

Enterprise and government networks are heterogeneous by nature — firewalls, routers, switches and cloud security groups from dozens of vendors, each required to comply with CIS Benchmarks, NIST SP 800-53, DISA STIGs and ISO/IEC 27001. Network devices are the gatekeepers of data and simultaneously the most common point of misconfiguration. Today the industry addresses this either through manual checklist auditing, or through expensive vendor-locked suites that do not flex across a mixed estate.

The two hard parts

- **Syntactic diversity.** The same security control is expressed differently on every platform. A password policy on Cisco IOS looks nothing like the same policy on a Juniper SRX, and neither resembles a PAN-OS XML export or a cloud security group.
- **Scalability and adaptation.** The landscape is not static. White-box networking (SONiC), cloud-native security groups and newly inducted or indigenous hardware all break hard-coded parsers, because a parser cannot anticipate a syntax it has never seen.

What the problem statement explicitly asks for

Required capability	What it means in practice
Unified ingestion engine	Upload single or bulk configuration files from any device.
AI-powered training module	A GUI where an administrator teaches the system to parse an unseen vendor format by mapping command outputs to compliance parameters.
Multi-framework compliance engine	Evaluate against user-selected benchmarks: CIS, NIST, STIG, ISO.
Actionable intelligence and PDF reporting	Per-device report with device identification, Pass/Fail with risk severity, and device-specific step-by-step remediation CLI.
Vendor-agnostic scalability	Modular architecture supporting new vendors, standards and OS versions <i>without manual code modification for every update</i> .

The clause that defines the project. The PS requires that the system learn a new vendor's logic *without requiring backend code redeployment*. That single sentence rules out the approach every existing tool takes — where the vendor's own engineering team writes each new parser — and it is the requirement most solutions quietly skip. We have built the product around it.

2. Our solution

NIRIKSHAK ingests configurations from any vendor, normalises them into a single vendor-neutral security schema, evaluates that schema against four compliance frameworks with a deterministic rule engine, and returns evidence-linked findings with device-exact remediation. When it meets a structure it does not recognise, it does not guess — it asks the administrator, and permanently learns the answer.



The core mechanism: vendor packs

A **vendor pack** is a data file describing how one platform's syntax maps to the canonical schema. Packs ship built-in for known vendors. When parsing produces unrecognised lines, the system clusters them, uses embedding similarity against every labelled example it already holds to propose the three most likely security categories, and presents them in the training GUI. The administrator confirms or corrects; the confirmation compiles into a deterministic pattern appended to that vendor's pack, and is simultaneously added to the similarity index.

Two consequences follow. Support for a new vendor is a **data change, not a code release** — satisfying the PS clause above. And because confirmed examples enter a shared index, teaching vendor A measurably improves the suggestions offered for vendor B.

Coverage compounds.

Six architecture rules

1. **AI never issues a verdict.** A deterministic rule engine reading the canonical model decides every Pass/Fail. The model's only jobs are suggesting a field mapping for an unknown line, and phrasing explanations.
2. **Every parsed field carries value, confidence and evidence.** Evidence is a pointer to the exact source file and line. If we cannot cite the line, we do not make the claim.
3. **Low confidence abstains.** Below threshold a field is reported UNKNOWN and routed to training. It never becomes a guessed PASS or FAIL.

4. Remediation comes from a vetted snippet library keyed by vendor, OS family and rule — never model-generated. A hallucinated command pasted into a production device is the most damaging failure this system could produce.
5. Rules and vendor packs are data, not code. Adding a framework or a vendor is a configuration change.
6. Offline-first. Local CPU inference, secrets scrubbed before any model call, encryption at rest, full audit trail. Configuration files never need to leave the operator's network.

3. How the solution meets the problem statement

PS requirement	How NIRIKSHAK satisfies it
Unified ingestion engine	Bulk upload with automatic format and vendor detection across CLI text, XML and JSON exports — covering both scraped CLI output and native structured exports, which differ substantially even within one vendor.
Normalisation to a vendor-neutral schema	A canonical Security Baseline Model of security-relevant fields plus a structured ACL model. Deterministic templates resolve known platforms; the AI layer is applied only to the residue.
Deviation analysis against frameworks	Declarative YAML rulepacks evaluated by a deterministic engine. One underlying check is cross-mapped to its CIS, NIST, STIG and ISO control IDs, so a single ingestion produces evidence for all four.
Dynamic adaptation / interactive training	Unknown lines are clustered and ranked by frequency, presented with top-3 AI suggestions, confirmed by the administrator, and compiled into a versioned vendor pack. No redeployment, no restart.
Actionable intelligence and PDF reporting	Per-device PDF containing device identification, Pass/Fail with severity, the source line proving each finding, and ordered remediation commands specific to that platform and version.
Vendor-agnostic scalability	Packs, rules and remediation snippets are all data. New vendors, frameworks and OS versions are added without touching application code.

4. Beyond the base idea

A parser plus a checklist satisfies the letter of the problem statement. It does not produce something an auditor would rely on. Five additions close that gap; each is deterministic and explainable, because a security tool that cannot justify its output is worse than no tool.

Absence-aware evaluation

Most non-compliance is an *absent* line, not a present one. A hardening directive may be missing because it is the platform default, or because someone removed it — opposite conclusions from identical evidence. NIRIKSHAK maintains a per-OS capability and default model: if the platform supports a control and it is absent, we evaluate against the documented default; if we do not know whether that platform supports it, we abstain. This single distinction separates a usable audit from a misleading one.

Peer-baseline outlier detection

Across a fleet, the system reports devices that deviate from their own peer group — forty-seven switches with a logging host configured and three without. This is statistical and fully explainable, requires no model, and surfaces drift that no static checklist can express.

Semantic ACL analysis

Access lists are evaluated as interval logic over source, destination, protocol and port to detect shadowed, redundant and overly permissive rules. This is computation, not pattern matching, and brings a measure of formal-analysis rigour into a compliance tool.

Exposure-aware prioritisation

Severity alone is a poor ranking. A weak cipher on a management interface reachable from a user VLAN is not the same risk as the same cipher behind a deny-all ACL. Reasoning over the canonical model together with parsed ACLs turns a flat findings list into an ordered remediation queue.

Calibrated confidence and measured abstention

Raw similarity scores are not confidence. Scores are calibrated against a hand-labelled corpus so that a stated confidence corresponds to observed accuracy, which makes the abstention threshold defensible rather than arbitrary.

5. Deliberate restraint

This is a cybersecurity tool operating on configurations that describe an organisation's internal topology and access controls. Several capabilities that would demonstrate well are excluded on principle.

- **No automatic remediation.** The system generates commands, a rollback block and an impact assessment. A human applies them. An auto-remediating tool with a parsing defect is an outage generator.
- **Lockout-risk ordering.** Remediation is ordered by dependency and service impact is flagged — disabling an insecure management protocol before its replacement is verified would strand the operator outside their own device.
- **No live device access or network scanning.** Offline configuration exports only. This keeps the deployment footprint minimal and requires no credentials.
- **No conversational interface over configurations.** A chat surface invites users to treat model output as a security judgement,

which contradicts rule 1.

- **No model fine-tuning.** A fine-tuned model is a static artefact requiring retraining for each new vendor — precisely what the problem statement rules out.

Confidentiality by construction. Secrets and credential-adjacent strings are scrubbed before any inference call. Configurations are encrypted at rest, access to raw files is role-separated from access to findings, and every AI suggestion and human correction is recorded in a hash-chained audit log. Because verdicts derive from the canonical model rather than from raw text interpreted by a model, content inside a configuration file cannot influence a compliance outcome.

6. How we prove it works

Accuracy is reported as a measurement, not a claim. A hand-labelled ground-truth set across the sample corpus scores the system on:

- **Precision and recall per canonical field** — is normalisation actually correct.
- **Correct-abstention rate** — cases where the system declined to answer and the control genuinely was not determinable. Honest uncertainty is scored as success.
- **Wrong-confident rate** — confident answers that were incorrect. This is the figure that must remain near zero, and the one we publish.
- **Generalisation** — one vendor is held out entirely, and we measure the proportion of its commands for which the similarity layer proposes the correct field within its top three suggestions. This is direct evidence that coverage compounds.

7. Implementation summary

Layer	Technology
Backend	Python 3.11, FastAPI, SQLite; queue-ready for batch audits
Deterministic parsing	TextFSM with ntc-templates, Netmiko / NAPALM, lxml for XML and JSON exports
AI layer	sentence-transformers on CPU with FAISS similarity search; local LLM via Ollama for offline operation
Compliance engine	Declarative YAML rulepacks with cross-framework control mapping
Interface	React and Tailwind dashboard; low-code training GUI; bulk upload
Reporting	Jinja2 with WeasyPrint for per-device PDF; hash-chained audit log

Efficiency. Configuration lines are hashed and their parse results cached, so identical lines across a large fleet are resolved once. Embedding inference runs only on the residue left after deterministic matching — typically a small fraction of a file. Unknown lines are clustered and frequency-ranked so an administrator confirms a few dozen patterns rather than reviewing hundreds of lines.

Hardware. The full pipeline runs on a standard 8 GB laptop with no GPU and no paid API. The same codebase scales to GPU-hosted local models, a production vector store and worker queues for fleet-scale deployment.

8. Deliverables

Source code repository; README with setup instructions; architecture document (maximum two pages); demo video (maximum two minutes); technical presentation (maximum five slides).